

Artificial Intelligence (CS-203)

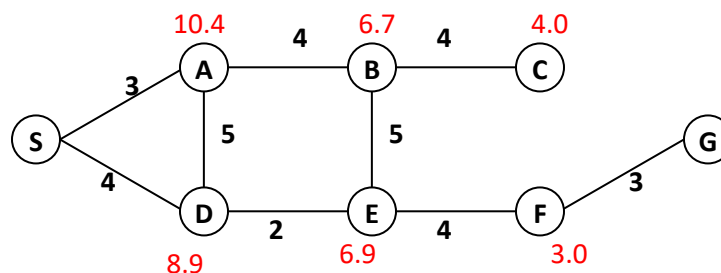
Objectives:

- Optimal path Search Strategies
- ✓ Branch and Bound Search
- ✓ A* Search

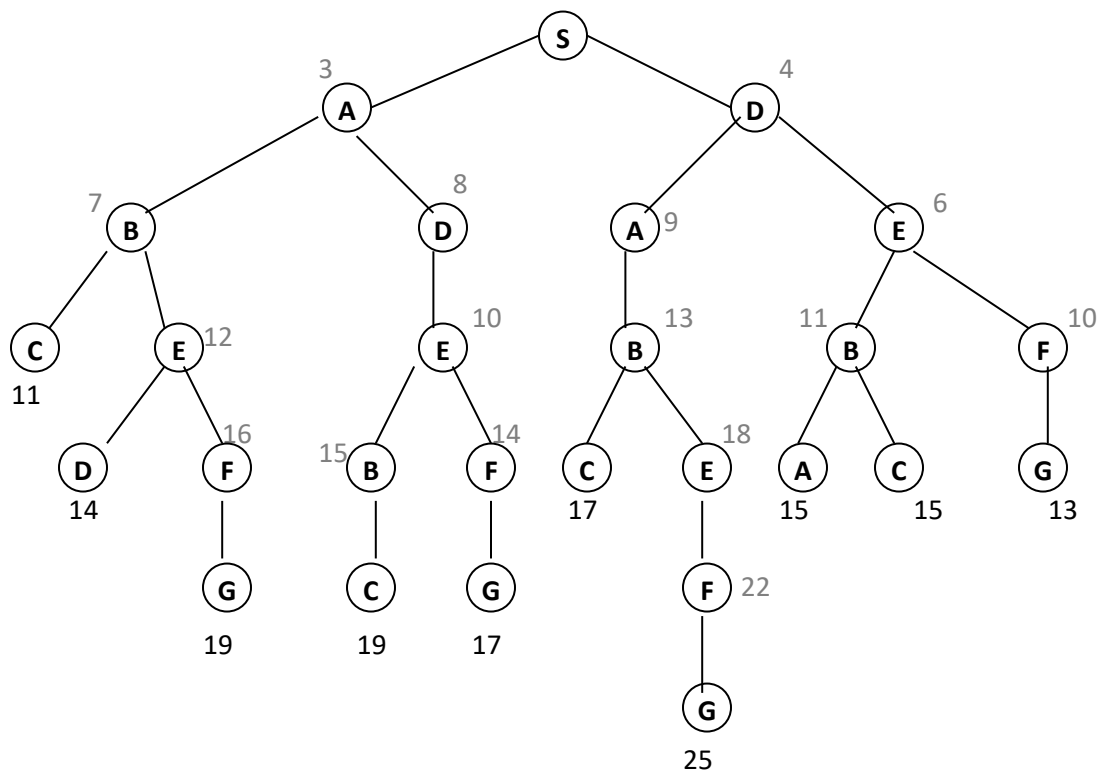
OPTIMAL PATH SEARCHING TECHNIQUES

We shall be using same graph and the tree for tracing optimal path search techniques which we used for some path searching algorithms last week.

Finding paths can be exemplified by having a net of cities connected by highways, such as the net shown in the figure below:



The objective is to begin at city S, the starting point, and end at city G, the final goal. A sample path could be any one of the following: S-D-E-F-G, S-A-D-E-F-G-, S-A-B-E-F-G, S-D-A-B-E-F-G, and so on. Whereas the optimal path (or the shortest path) is only S-D-E-F-G.



Branch-and-bound Search

Branch-and-bound search tries to discover the optimal path by using the least cost partial path (where the cost is determined by adding the distances traveled along the path, e.g. S_A_B is 7, S_D_E is 6, S_D_A is 9). The scheme works as follows: During search there are many incomplete paths contending for further consideration. The shortest one is extended one level, creating as many new incomplete paths as there are branches. These new paths are then considered along with the remaining old ones, and again, the shortest is extended. This repeats until the destination is reached along some path. Since the shortest path was always chosen for extension, the path first reaching the destination is certain to be optimal. The specific algorithm followed by branch-and-bound search is (where the **queue** is of partial paths not nodes):

- 2b If the first path does not reach the goal node:
 - 2b1 Remove the first path from queue.
 - 2b2 Form new paths from the removed path by extending one step.
 - 2b3 Add the new paths to the queue.
 - 2b4 Short the queue by cost accumulated so far, with least-cost paths in front.

```

      ((S))
        3      4
      ((SA) (SD))
        4      7      8
      ((SD) (SAB) (SAD))
        6      7      8      9
      ((SDE) (SAB) (SAD) (SDA))
        7      8      9      10      11
      ((SAB) (SAD) (SDA) (SDEF) (SDEB))
        8      9      10      11      11      12
      ( (SAD) (SDA) (SDEF) (SDEB) (SABC) (SABE))
        9      10      10      11      11      12
      ((SDA) (SDEF) (SADE) (SDEB) (SABC) (SABE))
        10      10      11      11      12      13
      ((SDEF) (SADE) (SDEB) (SABC) (SABE) (SDAB))
        10      11      11      12      13      13
      ((SADE) (SDEB) (SABC) (SABE) (SDEFG) (SDAB))
        11      11      12      13      13      14      15
      ((SDEB) (SABC) (SABE) (SDEFG) (SDAB) (SADEF) (SADEB))
        11      12      13      13      14      15      15      15
      ((SABC) (SABE) (SDEFG) (SDAB) (SADEF) (SDEBA) (SDEBC) (SADEB))
        12      13      13      14      15      15      15
      ((SABE) (SDEFG) (SDAB) (SADEF) (SDEBA) (SDEBC) (SADEB))
        13      13      14      14      15      15      15      16
      ((SDEFG) (SDAB) (SABED) (SADEF) (SDEBA) (SDEBC) (SADEB) (SABEF))
      Answer = (SDEFG)
  
```

Branch-and-bound Search with Dynamic Programming

Branch-and-bound search has a tendency to expand nodes, which have already been visited through a shorter path. For example node **D** was visited twice in the standard branch-and-bound search implementation (see figure given earlier). Let's look at this problem again, The root node **S** has been expanded into **A** and **D**. For the moment we use no underestimates for remaining path length. Since the path from **S** to **A** is shorter than the path from **S** to **D**, **A** has

been expanded also, leaving three paths: S-A-B, S-A-D and S-D. Thus the path S-D will be the next path extended, since it is the partial path with the shortest length.

But what about the path S-A-D? will it ever make sense to extend it? Clearly it will not. Since there is one path to **D** with length 4, it cannot make sense to work with another path to **D** with length 8. The path S-A-D should be forgotten forever. This is the underlying algorithm followed by dynamic programming. Consequently, the principle holds that when looking for the best path from **S** to **G**, all paths from **S** to any intermediate node, **I**, other than the minimum-length path from **S** to **I**, can be ignored. The specific algorithm of branch-and bound with dynamic programming is:

2b If the first path does not reach the goal node:

- 2b1 Remove the first path from queue.
- 2b2 Form new paths from the removed path by extending one step.
- 2b3 Add the new paths to the queue.
- 2b4 Short the queue by cost accumulated so far with least-cost paths in front.
- 2b5 If two or more paths reach a common node, delete all those paths except the one that reaches the common node with minimum cost.

The following tracing shows the effect of using dynamic programming principle, together with branch-and-bound search, on the map traversal problem. Four paths are cut off quickly, leaving only the dead-end path to node **C** and the optimal path, S-D-E-F-G.

```

      ((S))
      3      4
    ((SA) (SD))
      4      7      8
    ((SD) (SAB)) (SAD)
      6      7      9
    ((SDE) (SAB)) (SDA)
      7      10     11
    ((SAB) (SDEF)) (SDEB)
      10     11     12
    ((SDEF) (SABC)) (SABE)
      11     13
    ((SABC) (SDEFG))
      13
    ((SDEFG))
Answer = (SDEFG)

```

Branch-and-bound Search with Underestimates

In some cases, branch-and-bound search can be improved greatly by using guesses about distance remaining as well as facts about distances already traveled. In general, however, guesses are not perfect, and a bad overestimate somewhere along the true optimal path may cause the system to wander off the optimal path permanently. On the other hand, underestimates cannot cause the right path to be overlooked. An underestimate of the distance remaining yields an underestimate of total path length, u (total path length):

$$u(\text{total path length}) = d(\text{already traveled}) + u(\text{distance remaining})$$

where $d(\text{already traveled})$ is the known distance already traveled and where $u(\text{distance remaining})$ is an underestimate of the distance remaining. The algorithm followed by this modified scheme is:

- 2b If the first path does not reach the goal node:
 - 2b1 Remove the first path from queue.
 - 2b2 Form new paths from the removed path by extending one step.
 - 2b3 Add the new paths to the queue.
 - 2b4 Short the queue by sum of cost accumulated so far and a lower-bound estimate of the cost remaining, with least-cost paths in front.

Tracing of the example problem for this searching technique is as follows:

```

      ((S))
      12.9  13.4
    ((SD) (SA))
      12.9  13.4  19.4
    ((SDE) (SA) (SDA))
      13.0  13.4  17.7  19.4
  ((SDEF) (SA) (SDEB) (SDA))
      13.0  13.4  17.7  19.4
((SDEFG) (SA) (SDEB) (SDA))
Answer = (SDEFG)

```

A* Search

The A* search is branch-and-bound search, with an estimate of remaining distance, combined with the dynamic programming principle. If the estimate of remaining distance is a lower bound on the actual distance, then A* produces optimal solutions. Generally, the estimate may be assumed to be a lower bound estimate, unless specifically stated otherwise, implying that A*'s solutions are normally optimal. The algorithm followed by A* search is:

- 2b If the first path does not reach the goal node:
 - 2b1 Remove the first path from queue.
 - 2b2 Form new paths from the removed path by extending one step.
 - 2b3 Add the new paths to the queue.
 - 2b4 Short the queue by sum of cost accumulated so far and a lower-bound estimate of the cost remaining, with least-cost paths in front.
 - 2b5 If two or more paths reach a common node, delete all those paths except the one that reaches the common node with minimum cost.

```

      ((S))
      12.9  13.4
    ((SD) (SA))
      12.9  13.4
    ((SDE) (SA))
      13.0  13.4  17.7
  ((SDEF) (SA) (SDEB))
      13.0  13.4  17.7
((SDEFG) (SA) (SDEB))
Answer = (SDEFG)

```